

Electronics and Robotics Guild Hockey RL Competition

Getting started

A recruitment filter, not a Kaggle board. Beat the weak bot, survive the round-robin, then explain what you built.

Do this in the next fifteen minutes

1. Clone the repo. Use **Python 3.10–3.12**.
2. `python3.12 -m venv .venv && source .venv/bin/activate`
3. `pip install -r requirements.txt`
On macOS, if `box2d-py` fails to build: `brew install swig` and retry.
4. `python submission_template/agent.py` — should print `action shape: (4,)`
5. `python scripts/validate_submission.py submission_template`

Copy `submission_template/` to your working folder. The random policy is a load check, not a competitor.

1 What this is

You train a reinforcement-learning agent to play a two-player, continuous-control hockey game built on Box2D (martius-lab/hockey-env). You submit a small folder. We run your agent in isolation against baselines, then against everyone else.

Leaderboard rank is the *entry ticket*. The actual recruitment filter is a short viva with the people who wrote agents that survived Phase 2. We will ask why you chose the algorithm, how you shaped the reward, and what broke.

2 How you are judged



Phase 1	Your agent plays <code>weak_bot</code> and <code>strong_bot</code> only. Scores are public. Beating the weak bot is the first real signal; a 30–70 loss to the strong bot is still respectable. The public board is not the final ranking.
Phase 2	Standings are wiped. Every valid submission plays every other. Ranking is by Elo (start 1000, $K = 32$), not raw win rate. Agents that only memorised the weak bot usually collapse here.
Viva	Top 5–8 teams, about ten minutes. Reward design, failure modes, algorithm choice. If you cannot explain a curve in your writeup, the rank will not save you.

3 The contract

Every submission is a Python file named `agent.py` that defines exactly this class. `__init__` loads weights. `act` is called every physics step. Do not train in `__init__`.

```

class Agent:
    def __init__(self):
        # Load weights. Keep this fast.
        pass

    def act(self, observation: np.ndarray) -> np.ndarray:
        # observation: shape (18,)
        # return: shape (4,), values in [-1, 1]
        raise NotImplementedError

```

Eval instantiates `Agent` once per match process, then calls `act` thousands of times. A single call that exceeds **100 ms** fails validation. Exceptions in `act` are a forfeit for that episode.

3.1 Observation — 18 floats, player-first

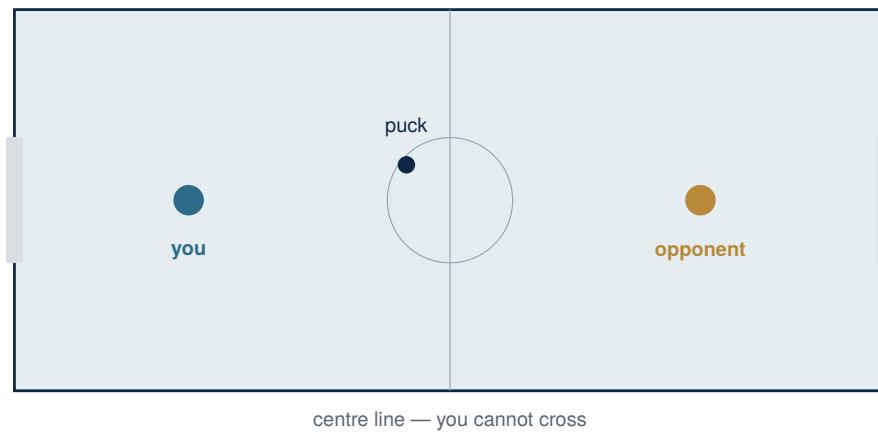
The vector is what `HockeyEnv` actually returns. Index 0 is *you* when you are player 1. When you are player 2, eval feeds the *mirrored* vector from `env.obs_agent_two()` so you still look like player 1. If you self-play, you must do the same.

Index	Slice	Meaning
0–1	[0:2]	Player 1 position (x, y)
2	[2]	Player 1 angle
3–4	[3:5]	Player 1 velocity (v_x, v_y)
5	[5]	Player 1 angular velocity
6–7	[6:8]	Player 2 position (x, y)
8	[8]	Player 2 angle
9–10	[9:11]	Player 2 velocity
11	[11]	Player 2 angular velocity
12–13	[12:14]	Puck position (x, y)
14–15	[14:16]	Puck velocity
16	[16]	Player 1 has-puck timer
17	[17]	Player 2 has-puck timer

3.2 Action — 4 floats, clipped to [-1, 1]

i	Control	Notes
0	Force x	Along the length of the rink
1	Force y	Lateral
2	Torque	Rotation of the mallet
3	Shoot	Fires when the value is > 0.5

Clip before you return. Unclipped actions make Box2D behave oddly. A NORMAL episode is **250** steps; treat both `terminated` and `truncated` as “game over”.



4 Training

`submission_template/train.py` fits Stable-Baselines3 SAC against the built-in *weak* opponent for 500k steps, using `HockeySingleAgentEnv`. That wrapper steps the opponent internally so SB3 sees a single-agent gymnasium env.

```
python submission_template/train.py --timesteps 500000 --output my_agent
```

This is a **pipeline proof**, not a good agent. Things that actually move the needle: reward shaping (closeness to puck, shot timing, not spinning), a curriculum (weak → strong → self-play), longer training, and not overfitting Phase 1.

Load the zip next to `agent.py`:

```
from stable_baselines3 import SAC

class Agent:
    def __init__(self):
        self.model = SAC.load("my_agent.zip")

    def act(self, observation):
        action, _ = self.model.predict(observation, deterministic=True)
        return np.asarray(action, dtype=np.float32)
```

5 What to submit

A folder (zip or git link — we will tell you which) containing:

File	Required	
<code>agent.py</code>	yes	The class above
model weights	yes	Whatever <code>__init__</code> loads
<code>requirements.txt</code>	yes	Extra pip deps, pinned if it matters
<code>writeup.md</code>	yes	See below. No writeup = disqualified

A template lives in `submission_template/writeup.md`. Four sections, all mandatory:

1. **Algorithm** — what you trained and why it fits this game.
2. **Reward** — the terms you used and the behaviour each one was for.
3. **Something that broke** — one failure, the evidence, the fix.

4. Training curves — labelled plots, not a screenshot of a terminal.

Check locally before you send it:

```
python scripts/validate_submission.py path/to/your_folder
```

Exit code 0 means the file loads, `act` returns shape `(4, )`, and inference stays under 100 ms. It does *not* mean the writeup is good.

6 How evaluation works

- Each match is 20 episodes (unless we say otherwise), sides swapped every episode so you are not stuck on the left.
- Player 2 always receives `obs_agent_two()`. You never see the raw player-1 vector when you are on the right.
- The match runs in a **subprocess**. A hang is killed; unfinished episodes count as draws and the match is marked `timeout`.
- Default wall-clock budget is 120 seconds per match. Keep `act` cheap.
- Elo starts at 1000 for every agent, including the built-in bots. The leaderboard sorts by Elo.

Five ways to silently lose

1. Feeding player 2 the raw observation instead of the mirrored one (in your own self-play).
2. Returning an action that is not shape `(4, )` or contains NaNs.
3. Training inside `Agent.__init__` — it will time out before the first face-off.
4. Ignoring `truncated`. Time-expired games are draws, not wins.
5. A 200 ms policy network. Validation will reject you; eval would too.

7 What “good” looks like

A submission we take seriously usually:

- Beats `weak_bot` consistently.
- Has *some* plan against `strong_bot` — losing 30–70 is fine; spinning in place is not.
- Does not degenerate (never moving, always shooting, orbiting its own goal).
- Comes with a writeup that could be read aloud in the viva without embarrassment.

If the SAC template already “kind of works,” that is the starting line. Self-play, opponent sampling, and a reward you can defend are the actual contest.

Questions go to the ERG organisers. Rebuild this PDF with `make -C guide pdf`.